

Code Generation Competition: 16 Proprietary vs. Open-Source LLMs & Iterative Learning Based on FDA Adverse Event Reporting System

Kevin Kawchak 

Chief Executive Officer

ChemicalQDevice

San Diego, CA

December 22, 2025

kevink@chemicalqdevice.com

ABSTRACT

Few effective goal-oriented iterative LLM code benchmarking studies exist. Successive high dimensional and complex problem improvements are desired versus conventional code assessments. Inspired by a recent CodeClash study, this tournament focuses primarily on the goal of generating functions to obtain a perfect competition task score based on three recent FDA FAERS files. Here, Opus 4.5 Extended was primarily utilized to build a novel Python evaluation engine measuring LLM code pair correctness, methodology, code quality, and algorithm effectiveness against a fixed reference standard and head-to-head. The notebook then automated Code A and Code B grading, and outputted their answers and reference standard of drug-reaction signals in csv files. The bracket was organized at scale: 16 LLMs - 8 proprietary LLMs on the left and 8 open-source LLMs on the right. The 8 Round 1 winners and corresponding notebooks were then re-introduced to each LLM with a competition prompt to generate the next round's code submission. Iterative learning in the form of improved final scores was observed for several Round 2 winners, which was based on its prior round competition code, competitors' code, and results. Gpt-5.2-pro and Gemini 2.5 Pro API were effective at iterative learning on the FAERS dataset goal; while Kimi K2 Thinking saw the biggest single round score increase at +0.405. Contestant models were from xAI, OpenAI, Gemini, Claude, DeepSeek, Kimi, GLM, MiniMax, and Qwen manufacturers.

Keywords Goal Oriented Software Engineering · LLM Code Generation · Iterative Learning · FDA FAERS

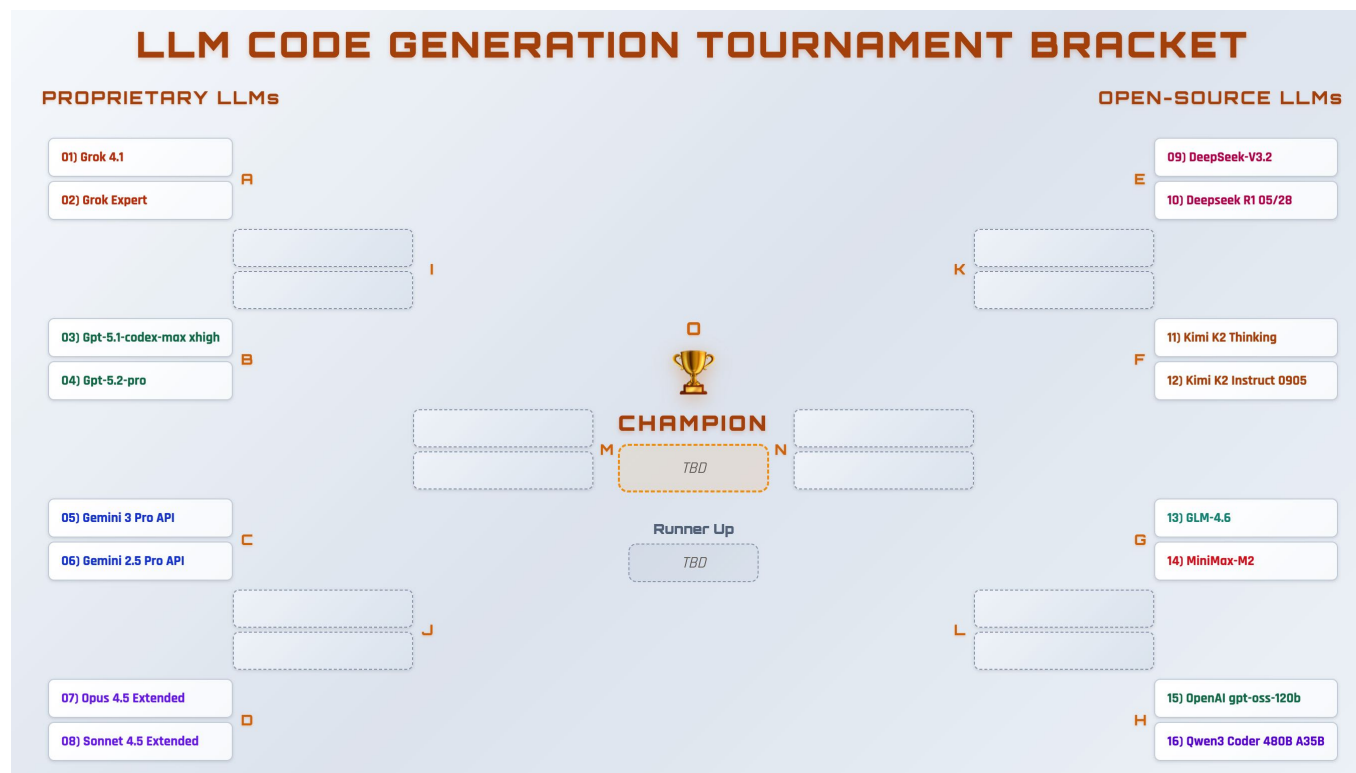


Figure 1: 16 LLM Single-Elimination Code Generation Bracket with Iterative Learning Process Throughout Rounds Based on FDA Adverse Event Reporting System (FAERS)

Table of Contents

1	Introduction	3
1.1	Human Code Iteration	3
1.2	AI Code Iteration	3
1.3	LLM Code Iteration	3
2	Methods	4
3	FDA FAERS Data Files	5
4	Results	5
4.1	FAERS & Rounds 1-3	5
4.2	Round 4 Code Results	5
4.3	Round 4 Notebook Results	7
4.4	Reference-Based Scoring Metrics	9
4.5	Round 4 Final Results	11
5	Discussion	13
5.1	Academic Paper References	13
5.2	Difficulty Benchmarking	14
5.3	Difficulty Benchmarking Standards	14
6	Limitations and Future Work	16
7	Conclusions	16
8	Prompts	17
9	Data Availability	20
10	References	20
11	Acknowledgments	23
12	Ethical disclosures	23
13	Rights and permissions	23
14	Cite this article	23

1 Introduction

1.1 Human Code Iteration

Humans have been iterating on code for many decades, continually evolving their techniques to obtain maximum efficiency [1, 2]. What started as labor-intensive writing and debugging in low level languages has evolved into higher-level language automation [3]. Additionally, interactive debugging and profiling tools have made code iteration cycles faster. For example, a programmer can run a Python script, profile its performance, and identify bottlenecks in seconds. Similarly, a programmer’s choice of top performing algorithms or data structures can yield large performance gains. Humans can effectively leverage their understanding of the problem domain and context, iterating through memory analyzers to reduce bloat. For instance, developers systematically optimized the interpreter in version Python 3.11, yielding an average 1.22× speedup over Python 3.10, with some benchmarks running 10–60% faster [4].

The 2020s introduced structured productivity frameworks like DevOps Research and Assessment’s DORA metrics (Deployment frequency, Lead time, Change failure rate, Mean time to restore) [5], and Microsoft’s SPACE framework (Satisfaction, Performance, Activity, Communication, Efficiency) [6]. However, humans have been found to be limited to code review effectiveness peaks at under 400 lines per session [7]. Global optimization is extremely challenging for humans to perform; and optimizing an entire large codebase or complex system is often beyond what a single human (or team) can manage.

- Manual optimization is time-consuming and labor-intensive
- Humans tend to stop once “fast enough” performance is reached due to fatigue
- Diminishing returns: A lot of effort for small gains in obtaining last 3% improvement

1.2 AI Code Iteration

AI code iteration utilizes automated algorithms or machine intelligence to iteratively refine code. This includes search-based optimization, genetic programming, automated program repair, reinforcement learning, and ML-guided compilers [8]. DeepMind’s 2022 AlphaTensor project applied reinforcement learning to discover new matrix multiplication algorithms, finding math formulas more efficiently and reducing the number of operations needed [9, 10]. DeepMind’s 2023 AlphaDev also utilized reinforcement learning in treating algorithm discovery as a game. AlphaDev found new sorting algorithms that surpassed decades-old human benchmarks, and sorting routines were up to 70% faster for short sequences; while large sequences saw a 1.7% speed improvement in the C++ standard sorting [11, 12]. Bowtie2 by Langdon, W. et al. optimized the DNA sequence alignment program based on a complex 50,000-line bioinformatics tool. By evolving the C++ code, the AI produced a new version that ran on average 70× faster [13].

Conversely, AI code iterations are rigid, in that the code must be machine-readable, and the improvement goal must be encoded as a fitness function or reward. This lack of flexibility means these systems only accept certain input formats (e.g. a C++ function plus a test suite). AI systems (pre-LLM) do not provide rationales or high-level explanations for the changes to users; they just output the new code. This opacity means developers might be hesitant to trust and deploy such code. In practice, some AI-discovered solutions have to be carefully reviewed and proved equivalent to the original [14].

- AI code iteration cannot understanding natural language intent or high-level goals
- Concrete metric or formal specification requires expertise for success at scale
- AlphaDev required framing as sorting assembly instructions with a defined reward [12]

1.3 LLM Code Iteration

LLM code iteration builds on AI code iteration, but now with mixed language and high contextual awareness capabilities. LLMs have been trained on large portions of internet code and discussions (GitHub repositories [15], Stack Overflow Q&A [16], etc.). DeepMind’s 2025 AlphaEvolve used algorithmic AI agents combining evolutionary search with LLM-guided code generation to iteratively evolve algorithms [17, 18]; producing a hashing algorithm 30% faster than a widely-used human-designed hash. Contemporary Opus 4.5 is “the coding and agentic workflow leader”, scoring 74.4% on a stringent SWE-bench to iteratively plan, code, and refine [19].

OpenAI Gpt-5.2’s key strength is speed: Gpt-5.2 processes about 187 tokens/second, which is 3.8× faster than Claude which makes iterations faster [20]. Gemini 3 Pro’s focus on multimodal input at 1 million token context window can iterate faster through entire codebases in a single prompt [21]. The recent DeepSeek V3.2 matches Western models on many benchmarks and boasts much lower cost to make workflows go farther at \$0.56/\$1.68 for 1M input and 1M output vs. \$5/\$25 for Opus [22] and \$1.75/\$14.00 for Gpt 5.2 [23] and \$2/\$12 for Gemini 3 [21].

- LLM interactive prompting and code refinement for generating revised versions
- Human in the loop collaboration for effectively noticing a wide range of issues [24]
- Encyclopedic knowledge of optimization tricks, library functions, and idiomatic improvements

2 Methods

Competition Notebook Development

The FDA Adverse Event Reporting System (FAERS) July - September 2025 Quarterly ASCII DEMO25Q3.txt, DRUG25Q3.txt, and REAC25Q3.txt data files were utilized in code executions. Opus 4.5 Extended utilized 12 prompts in a single conversation to develop the competition engine shown in Notebook_Generation. Prompt 01 contained author instructions and data file information to generate a notebook based on "Code_A" and "Code_B" competitors. Prompt 02 was aimed at improving Google Drive incorporation; and a sample of the dataset was included to improve applicability. ChatGPT 5.1 was utilized to determine 4 larger dataset competition tasks in Prompt 03a/3b [25]. Task A, later renamed Task 1, was ultimately chosen in Prompt 04a as the most relevant to FDA, in regards to LLM code identifying the "First calendar quarters where pair becomes a safety "signal" (PRR $\geq$ threshold AND count $\geq$ minimum cases)." Prompt 04 focused further on Task 1, with further competition task optimizations in Prompt 05. Additional improvements were made by Opus regarding the competition task being self contained in Prompt 06; with Output 06 USER_6th_FAERS_LLM_Competition_Task1_v2.ipynb being used in Round 0 LLM code generations.

LLM Competition Code Generations

Opus was then instructed to design a prompt based on the Output 06 notebook that generated a competition code with the goal of a perfect 1.0 final score based on correctness, methodology, code quality, and algorithmic efficiency, seen in Prompt 07. This prompt was used to assist the author's Round_1_Prompt to then generate LLM Round 0 code for 1st Round competition. Additional optimizations afforded an exact competition task; Code A, and Code B answers to Drive were achieved in Prompt 08. The reference solution correctness metric was further optimized in Prompt 09. Opus then recalculated the correctness score of each code submission based on the reference solution in Output 09. Precautions were taken to assure the notebook corresponded to LLM code generation outputs in Prompt 10, which included transitioning to reference-based scoring using pair F1 score, PRR correlation, quarter match, and structure. Errors and Drive issues were fixed in Output 11 and Output 12, with the final Output 12 _1RD_Tournament_FAERS.ipynb being used as the competition engine in Round 1.

Rounds 1-4 Code Competition

First round generated competition code was based on the earlier USER_6th_FAERS_LLM_Competition_Task1_v2.ipynb correctness score calculation, while the newer _1RD_Tournament_FAERS.ipynb was used to generate 2nd round code. Code pairs replaced existing _1RD_Tournament_FAERS.ipynb Section 5 code of each notebook running on a T4 GPU. Model thoughts (if available), outputs, and run times were included in Round_1.docx. Python notebook JSON code was provided to Gpt and open-source models where file uploads were not allowed. First round pairs A-H yielded 8 scoring results and up to 9 Drive outputs per notebook. Most importantly, 01_reference_solution.csv provided the exact 4 line solution, while Code A and Code B outputs either matched or varied from the reference solution. If a function did not execute, no file was outputted for that competitor. The highest LLM total weighted scores from Section 6 advanced to Round 2. Multi_Round_Prompt was used for generating Round_2, Round_3, and Round_4 competitor code; which all had minor "instructions for improvement" modifications compared to Round_1. Rounds 2-4 were run on a v6e-1 TPU, and based on each contestant's prior code competition notebook. For instance, Round 1 winner Grok Expert utilized Multi_Round_Prompt and its prior round A_1RD_Tournament_FAERS.ipynb to generate its Round 2 competitor code; as did Gpt-5.2-pro use Multi_Round_Prompt and its B_1RD_Tournament_FAERS.ipynb to generate its Round 2 code. At the end of the 4th round, a champion was determined based on the round's highest total weighted score.

LaTeX source code, notebook generation inputs & outputs, competition notebooks for each round, and each round's output are organized in the Data Availability section, and available in supplementary files. The author wrote the abstract, methods, results text, discussion, prompts (except meta prompts), limitations and future work, and conclusion; and was assisted by ChatGPT 5.2 Thinking Deep Research for the introduction. The remaining sections were either derived from data files, notebooks, or generated using LLMs. All visual elements were generated by Opus 4.5 Extended based on result data or notebooks. Google Colab [26] was used for notebook code editing, Visual Studio Code [27] was used for code viewing and edits. Google Docs [28] was used for documenting notebook generation inputs, thoughts, and outputs; Docs was also used for managing LLM generated thoughts and code outputs. All open-source models were run in each of the models' Fireworks.ai's "Try" playground page with standard settings, except for a 16,384 Max Tokens modification. All proprietary chat and API playground inference LLMs were executed in MacOS 14.5 (23F79) [29], Chrome Version 142.0.7444.162 (Official Build) (arm64) [30]. Total AI cost was approximately \$78.06. ChatGPT 5.1 and ChatGPT 5.2 were accessed through the OpenAI Plus plan chat interface as research assistants [25, 31].

(8) Proprietary LLMs: Chat/API Playground Interfaces

- **Grok:** xAI Grok 4.1 and Grok Expert were accessed through the SuperGrok website chat interface [32, 33]
- **Gpt:** Gpt-5.1-codex-max xhigh, Gpt-5.2-pro were accessed through platform.openai.com API "Chat" website [25, 31, 34]
- **Gemini:** Gemini 3 Pro, 2.5 Pro were accessed through API Key aistudio.google.com. Settings: Temp=1, Thinking level = High (3 Pro), Budget = 32768 (2.5 Pro), Tools = Off, Safety settings = Off, Output length = 65536, Top P = 0.95 [21, 35, 36]
- **Claude:** Opus 4.5 Extended, Sonnet 4.5 Extended were accessed through the Claude Pro plan chat interface [22, 37, 38]

(8) Open Source LLMs: Fireworks AI Defaults & 16,384 Max Tokens

- **DeepSeek:** DeepSeek-V3.2, R1 05/28 were accessed through the Fireworks.ai inference service [39, 40, 41]
- **Kimi:** Kimi K2 Thinking, Instruct 0905 were accessed through the Fireworks.ai inference service [42, 43, 41]
- **GLM:** GLM-4.6 was accessed through the Fireworks.ai inference service [44, 41]
- **MiniMax:** MiniMax-M2 was accessed through the Fireworks.ai inference service [45, 41]
- **OpenAI:** OpenAI gpt-oss-120b was accessed through the Fireworks.ai inference service [46, 41]
- **Qwen:** Qwen3 Coder 480B A35B was accessed through the Fireworks.ai inference service [47, 41]

3 FDA FAERS Data Files

FDA Adverse Event Reporting System (FAERS)													
DEMO25Q3: Demographic Data, First 3 Entries Shown													
primaryid	caseid	i_f_code	mfr_sndr	age	age_cod	sex	wt	wt_cod	rept_cod	occp_cod	reporter_country	occr_country	
100289774	10028977	F	GLAXOSMITHKLINE	53	YR	M			EXP	MD	IN	IN	
1005762123	10057621	F	ROCHE	57	YR	M	53	KG	EXP	CN	CA	CA	
1006831310	10068313	F	PFIZER	69	YR	F	72	KG	PER	CN	US	US	
DRUG25Q3: Drug Information 7 Columns Not Shown													
primaryid	caseid	drug_seq	role_cod	drugname	prod_ai	route	dose_vbm	dechal	rechal				
100289774	10028977	1	PS	DICLOFENAC POTASSIUM	DICLOFENAC POTASSIUM		UNK	Y	U				
100289774	10028977	2	C	ONDANSETRON	ONDANSETRON	Unknown	UNK	U					
100289774	10028977	3	C	TRAMADOL	TRAMADOL	Unknown	UNK	U					
REAC25Q3: Adverse Reactions, Column 4 Fields n/a													
primaryid	caseid	pt	drug_rec_act										
100289774	10028977	Fixed eruption											
100289774	10028977	Stevens-Johnson syndrome											
100289774	10028977	Toxic epidermal necrolysis											

Table 1: FAERS July - September 2025 Quarterly ASCII Data Files, 3 of 7 [48]

Rounds 1-4 Reference Solution (01_reference_solution.csv)					
Drug Name	Reaction	Emergence Quarter	Emergence PRR	Total Cases	Is Signal
DUPIXENT	Eczema	2025Q1	5.058	5	True
DUPIXENT	Pruritus	2025Q1	4.215	8	True
DUPIXENT	Condition aggravated	2025Q1	3.793	5	True
DUPIXENT	Dyspnoea	2025Q1	2.529	5	True

Table 2: Reference & Contestant Goal Output. Drug-Reaction Pairs Derived from DEMO, DRUG, REAC. First Calendar Quarters Where Pair Becomes a Signal (PRR $\geq$ Threshold AND Count $\geq$ Minimum Cases)

4 Results

4.1 FAERS & Rounds 1-3

Table 1 above represents a sampling of the static FDA Adverse Event Reporting System (FAERS) dataset used in rounds. Table 2 is the final reference solution that each competitor's function was seeking to output. After the `_1RD_Tournament_FAERS.ipynb` was finalized, head-to-head code matchups were conducted for Rounds 1-3, as detailed in the Methods section and in Figure 2. The winner of each round's Python notebook was used as the input along with `Multi_Round_Prompt` to enable iterative updates for subsequent rounds.

4.2 Round 4 Code Results

The 152 line `Gpt-5.2-pro Round 4 detect_signal_emergence_improved` function, which is based on its prior Round M notebook and `Multi_Round_Prompt` from Table 18 is provided in the following page to build drug-reaction pairs and compute PRR-like observed/expected per quarter. The function uses 9 algorithms for data cleaning/schema normalization, rule-based filtering, deterministic time-binning, relational join/entity resolution, frequency counting/contingency component estimation, observed-expected disproportionality, thresholding/decision rule, "first hitting time"/earliest-emergence selection, and post-processing/summarization.

Multi_Round_Prompt LLM Thoughts Addressed

- "Achieving a perfect score of 1.0 on all metrics is the goal"
- "To improve the quality from Code_A's score of 9.5 to 10 [prior Round 3 notebook's Gpt-5.2-pro code], I should add type hints, enhance docstrings, ensure vectorization, have a function count of at least three, and maintain a proper comment ratio"
- "I'm planning to create a function that includes type hints and input checking while avoiding external dependencies"

Competition Code Implements Useful Comments

1. "Matched the notebook Reference Solution exactly (per-quarter PRR via $\text{expected} = (\text{drug_cases} * \text{reac_cases}) / \text{total_cases}$, earliest-quarter emergence, PRR rounded to 3)."
2. "Hardened input handling (column normalization, strict FAERS event_dtquarter parsing, safe early exits with correct empty schema)."
3. "Kept performance high and stable (column subsetting, vectorized groupby/nunique + merges; no Python loops over rows)."

```

def detect_signal_emergence_improved(demo_df: pd.DataFrame,
                                     drug_df: pd.DataFrame,
                                     reac_df: pd.DataFrame,
                                     min_cases: int = 3,
                                     prr_threshold: float = 2.0) -> pd.DataFrame:
    """
    Improved implementation of Task 1 (TASK_1_SPEC).
    Improvements made:
    # 1. Matched the notebook Reference Solution exactly (per-quarter PRR via expected=(drug_cases*reac_cases)/total_cases, earliest-quarter emergence, PRR rounded to 3).
    # 2. Hardened input handling (column normalization, strict FAERS event_dt/quarter parsing, safe early exits with correct empty schema).
    # 3. Kept performance high and stable (column subsetting, vectorized groupby/nunique + merges; no Python loops over rows).
    Returns a DataFrame with columns:
    ['drug_name', 'reaction', 'emergence_quarter', 'emergence_prr', 'total_cases', 'is_signal']
    """
    required_out_cols = [
        "drug_name", "reaction", "emergence_quarter",
        "emergence_prr", "total_cases", "is_signal"
    ]
    def _empty() -> pd.DataFrame:
        return pd.DataFrame(columns=required_out_cols)
    def _norm_cols(df: pd.DataFrame) -> pd.DataFrame:
        if df is None or not isinstance(df, pd.DataFrame) or df.empty:
            return pd.DataFrame()
        out = df.copy(deep=False)
        out.columns = out.columns.astype(str).str.lower().str.strip()
        return out
    def _parse_quarter(demo: pd.DataFrame) -> pd.DataFrame:
        # Matches the notebook reference parsing behavior exactly.
        if demo.empty or ("primaryid" not in demo.columns) or ("event_dt" not in demo.columns):
            return pd.DataFrame(columns=["primaryid", "quarter"])
        d = demo[["primaryid", "event_dt"]].dropna(subset=["event_dt"]).copy()
        d["event_dt_str"] = d["event_dt"].astype(str).str.replace(".", "", regex=False)
        d = d[d["event_dt_str"].str.len() >= 6]
        d["year"] = d["event_dt_str"].str[0:4]
        d["month"] = pd.to_numeric(d["event_dt_str"].str[4:6], errors="coerce")
        d = d[d["month"].notna() & d["year"].astype(str).str.match(r"^\d{4}$", na=False)]
        d = d[(d["month"] >= 1) & (d["month"] <= 12)]
        d["quarter"] = d["year"].astype(str) + "Q" + ((d["month"] - 1) // 3 + 1).astype(int).astype(str)
        return d[["primaryid", "quarter"]]
    # --- Normalize inputs (robust to column casing/whitespace) ---
    demo = _norm_cols(demo_df)
    drug = _norm_cols(drug_df)
    reac = _norm_cols(reac_df)
    if demo.empty or drug.empty or reac.empty:
        return _empty()
    # --- Suspect drug filtering + normalization (uppercase, trimmed) ---
    need_drug_cols = {"primaryid", "drugname", "role_cod"}
    need_reac_cols = {"primaryid", "pt"}
    if not need_drug_cols.issubset(drug.columns) or not need_reac_cols.issubset(reac.columns):
        return _empty()
    drugs = drug.loc[drug["role_cod"].isin(["PS", "SS"]), ["primaryid", "drugname"]].copy()
    if drugs.empty:
        return _empty()
    drugs["drugname"] = drugs["drugname"].astype(str).str.upper().str.strip()
    # --- Parse quarters from DEMO ---
    demo_q = _parse_quarter(demo)
    if demo_q.empty:
        return _empty()
    # --- Merge DRUG + REAC + DEMO on primaryid (inner joins as reference) ---
    merged = (
        drugs.merge(reac[["primaryid", "pt"]], on="primaryid", how="inner")
        .merge(demo_q, on="primaryid", how="inner")
    )
    if merged.empty:
        return _empty()
    # --- Per-quarter component counts (unique primaryid) ---
    quarter_totals = (
        merged.groupby("quarter", sort=False)
        .agg(total_cases=("primaryid", "nunique"))
        .reset_index()
    )
    drug_quarter = (
        merged.groupby(["drugname", "quarter", sort=False] ["primaryid"]
        .nunique()
        .reset_index(name="drug_cases")
    )
    reac_quarter = (
        merged.groupby(["pt", "quarter", sort=False] ["primaryid"]
        .nunique()
        .reset_index(name="reac_cases")
    )
    pair_quarter = (
        merged.groupby(["drugname", "pt", "quarter", sort=False] ["primaryid"]
        .nunique()
        .reset_index(name="pair_cases")
    )
    analysis = (
        pair_quarter.merge(drug_quarter, on=["drugname", "quarter"], how="left")
        .merge(reac_quarter, on=["pt", "quarter"], how="left")
        .merge(quarter_totals, on="quarter", how="left")
    )
    # Reference PRR definition used in notebook:
    # expected = (drug_cases * reac_cases) / total_cases ; prr = pair_cases / expected
    analysis["expected"] = (analysis["drug_cases"] * analysis["reac_cases"]) / analysis["total_cases"]
    analysis["prr"] = np.where(analysis["expected"] > 0, analysis["pair_cases"] / analysis["expected"], 0.0)
    # --- Signal emergence (first quarter meeting threshold) ---
    signals = analysis[
        (analysis["prr"] >= float(prr_threshold)) & (analysis["pair_cases"] >= int(min_cases))
    ].copy()
    if signals.empty:
        return _empty()
    signals = signals.sort_values("quarter", ascending=True)
    first_emergence = signals.groupby(["drugname", "pt"]).first().reset_index()
    total_by_pair = (
        merged.groupby(["drugname", "pt", sort=False] ["primaryid"]
        .nunique()
        .reset_index(name="total_cases_all")
    )
    result_df = first_emergence[["drugname", "pt", "quarter", "prr"]].merge(
        total_by_pair, on=["drugname", "pt"], how="left"
    )
    result_df = result_df.rename(columns={
        "drugname": "drug_name",
        "pt": "reaction",
        "quarter": "emergence_quarter",
        "prr": "emergence_prr",
        "total_cases_all": "total_cases",
    })
    result_df["emergence_prr"] = result_df["emergence_prr"].astype(float).round(3)
    result_df["total_cases"] = result_df["total_cases"].fillna(0).astype(int)
    result_df["is_signal"] = True
    return (
        result_df[required_out_cols]
        .sort_values("emergence_prr", ascending=False)
        .reset_index(drop=True)
    )
# REQUIRED: Set result variable
result = detect_signal_emergence_improved(demo_df, drug_df, reac_df)
print(f"Found {len(result)} signals")

```

4.3 Round 4 Notebook Results

The notebook introduction shown in Figure 2 provides the cross-table temporal signal emergence detection competition task, scoring system, and output structure. It is important to note that correctness is based on a head-to-head comparison to 01_reference_solution.csv shown in Table 2. The following Table 3 shows time, speed, and cost benefits favoring the open-source gpt-oss-120b over the proprietary Gpt-5.2-pro. It was significant that the three Gpt-5.2-pro helper functions contributed to higher Code Quality score (0.95 vs. 0.90) and decisive error handling. The Round 4 results output provides each of the four reference-based scoring metrics (correctness, methodology, code quality, and algorithmic efficiency) of Code_A (Gpt-5.2-pro) and Code_B (OpenAI gpt-oss-120b) in Table 4. Gpt-5.2-pro (0.9775/1.0) correctness matched the reference solution (1.0/1.0), with a top methodology score (1.0/1.0); while OpenAI gpt-oss-120b (0.2050/1.0) failed to execute, but had a code quality score of 9.0/10. The multi-metric score framework is illustrated in Figure 3 and itemized with final weighted scores in Table 5-Table 9.

Head-to-Head LLM Code Competition Framework

FDA FAERS Pharmacovigilance Data Analysis

Task 1: Cross-Table Temporal Signal Emergence Detection

Overview

This notebook implements a structured head-to-head competition framework for evaluating LLM-generated Python code. Two competing code submissions (Code_A and Code_B) are executed against the same FDA Adverse Event Reporting System (FAERS) dataset, and their outputs are compared using standardized evaluation metrics.

Competition Structure

- **Task Definition (Section 3):** Fixed specification of what to solve - does NOT change between rounds
- **Code Submissions (Section 5):** LLM implementations that CAN be replaced each round
- **All rounds compete on the same Task 1** - only the implementations change

Scoring System

Correctness scoring compares submissions against the Reference Solution:

- **Pair Match Score:** % of drug-reaction pairs that match reference
- **Value Accuracy:** PRR value similarity for matched pairs
- **F1 Score:** Harmonic mean of precision and recall

Output Structure

Each run creates a **timestamped directory** containing:

- **Reference Solution:** The expected/ground truth answer for the task
- **Code_A Output:** Results from Code_A implementation
- **Code_B Output:** Results from Code_B implementation
- **Comparison Report:** Detailed comparison of outputs vs reference
- **Methodology Documentation:** Explanation of how each solution was formed
- **Competition Report:** Scores and evaluation metrics (JSON)
- **Peer Review Prompt:** For subsequent improvement rounds

Figure 2: Rounds 1-4 Competition Notebook Screenshot

Round 4 Code Generation Metrics (Single Iteration)		
Metric	Gpt-5.2-pro	gpt-oss-120b
Generation Time	2 min 49.37 sec	0 min 25.65 sec
Generation Speed	~19.5 tokens/sec	122.34 tokens/sec
Generation Cost	\$1.42	\$0.01
Output Tokens	3,292 T	~3,134 T (est.)
Input Tokens	46.8 KT	~41.6 KT (est.)
Lines of Code	127 LOC	142 LOC
Helper Functions	3 (_empty, _norm_cols, _parse_quarter)	0 (inline logic only)

Table 3: Proprietary vs. Open-Source Model Differences

Round 4 Results. Code_A = Gpt-5.2-pro. Code_B = OpenAI gpt-oss-120b

```

=====
🚀 STARTING HEAD-TO-HEAD COMPETITION
=====
Competition ID: FAERS_25Q3_COMPETITION
Task: Task 1: Cross-Table Temporal Signal Emergence Detection
Data Sample: 10%
Reference Solution: 4 signals
Output Directory: /content/drive/MyDrive/Colab Notebooks/Inputs/FAERS_LLM_Competition/results/run_20251212_215200
=====
🏁 COMPETITION ROUND: T1_TEMPORAL_SIGNAL_R1_038e016e
Task: Task 1: Cross-Table Temporal Signal Emergence Detection
Reference Solution: 4 signals
=====
Code_A hash: 9838497cf082
Code_B hash: ce3db45a915c
Executing Code_A...
Runtime: 0.123s | Success: True
Output: 4 rows
Executing Code_B...
Runtime: 0.001s | Success: False
=====
📊 ROUND RESULTS (Reference-Based Scoring)
=====
Code_A Score: 0.9775 🟢
  Correctness:      1.0000 (0.45)
    Pair F1:        1.0000 (P:1.00 R:1.00)
    PRR Correlation: 1.0000
    Quarter Match:  1.0000
    Matched Pairs:  4/4
  Methodology:      1.0000 (0.30)
  Code Quality:      9.50/10 (0.15)
  Algorithmic Effic.: 0.8500 (0.10)
Code_B Score: 0.2050 🚫 FAILED
  Correctness:      0.0000 (0.45)
    Pair F1:        0.0000 (P:0.00 R:0.00)
    PRR Correlation: 0.0000
    Quarter Match:  0.0000
    Matched Pairs:  0/0
  Methodology:      0.0000 (0.30)
  Code Quality:      9.00/10 (0.15)
  Algorithmic Effic.: 0.7000 (0.10)
🏆 WINNER: Code_A
=====

```

Table 4: Round 4 Competition Notebook Section 6.1 Results

4.4 Reference-Based Scoring Metrics

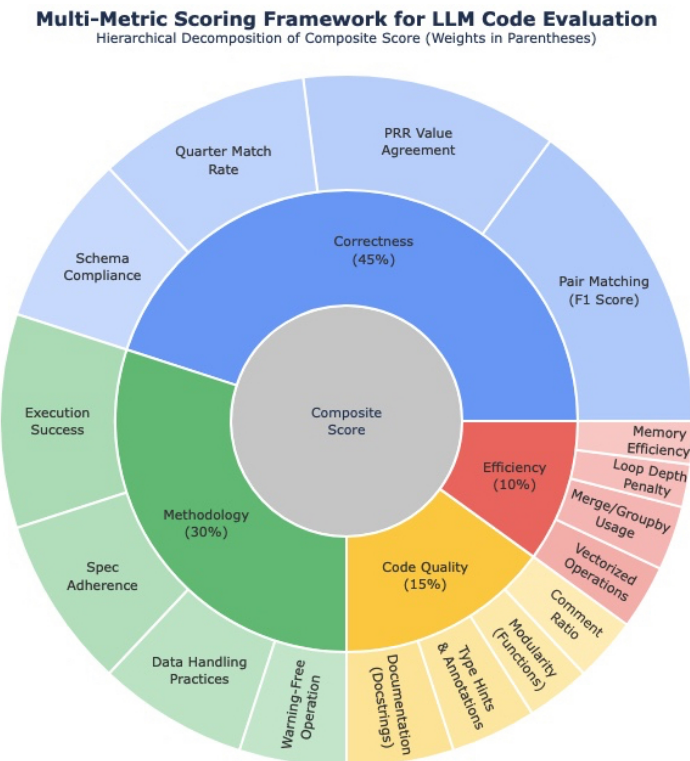


Figure 3: Reference-Based Weighted Scoring System

Correctness Scoring Breakdown (45% Weight)				
Subcategory	Weight	Gpt-5.2-pro	gpt-oss-120b	Assessment Method
Pair F1 Score	40%	1.0000	0.0000	Precision/Recall of pairs
PRR Correlation	30%	1.0000	0.0000	Pearson correlation
Quarter Match Rate	20%	1.0000	0.0000	Emergence quarter match
Structure Score	10%	1.0000	0.0000	Column schema compliance
CORRECTNESS TOTAL	100%	1.0000	0.0000	Sum of weighted scores
Contribution to Final	×0.45	0.4500	0.0000	Correctness × 0.45

Table 5: Correctness Scoring Formula = $0.40 \times F_1 + 0.30 \times r_{\text{PRR}} + 0.20 \times Q_{\text{match}} + 0.10 \times S_{\text{struct}}$

Methodology Scoring Breakdown (30% Weight)				
Subcategory	Points	Gpt-5.2-pro	gpt-oss-120b	Assessment Criterion
Execution Success (Base)	+0.70	✓ +0.70	✗ +0.00	Error-free execution
Warning Penalty	−0.10 each	0 warnings	N/A (failed)	Max −0.30 for 3+ warnings
Vectorized Operations	+0.10	✓ +0.10	N/A (failed)	Uses np.*, .str., .dt.
Function Count ≥ 2	+0.10	✓ +0.10	N/A (failed)	Modular code structure
Has Docstrings	+0.10	✓ +0.10	N/A (failed)	Triple-quote documentation
METHODOLOGY TOTAL	max 1.0	1.0000	0.0000	Capped at 1.0
Contribution to Final	×0.30	0.3000	0.0000	Methodology × 0.30

Table 6: Methodology Based on Execution Success, Code Practices. Base = 0.70 + (3) +0.10 Bonuses

Code Quality Scoring Breakdown (15% Weight)				
Subcategory	Points	Gpt-5.2-pro	gpt-oss-120b	Detection Method
Base Score	+5.00	+5.00	+5.00	All submissions start here
Has Docstrings	+1.50	✓ +1.50	✓ +1.50	""" or ''' present
Has Type Hints	+0.50	✓ +0.50	✓ +0.50	def func(x: Type)
Uses Vectorized Ops	+1.00	✓ +1.00	✓ +1.00	np.*, .str., .dt., .agg()
Function Count ≥ 1	+0.50	✓ +0.50	✓ +0.50	At least one def
Function Count ≥ 3	+0.50	✓ +0.50	✗ +0.00	Three or more def
Comment Ratio 10–30%	+0.50	✓ +0.50	✓ +0.50	# lines / code lines
CODE QUALITY TOTAL	/10	9.50	9.00	Capped at 10.0
Contribution to Final	$\times 0.015$	0.1425	0.1350	$(\text{Quality}/10) \times 0.15$

Table 7: Quality Scoring via Static Analysis, ≥ 3 Helper Functions = Bonus

Algorithmic Efficiency Scoring Breakdown (10% Weight)				
Subcategory	Points	Gpt-5.2-pro	gpt-oss-120b	Detection Pattern
Base Score	+0.50	+0.50	+0.50	All submissions start here
Uses Vectorized Ops	+0.15	✓ +0.15	✓ +0.15	np.where, .str., .dt.
Uses Merge/Join	+0.10	✓ +0.10	✓ +0.10	.merge() or .join()
Uses GroupBy	+0.10	✓ +0.10	✓ +0.10	.groupby()
Nested Loops ≥ 2	−0.15	✓ −0.00	✗ −0.15	Indent-based detection
Nested Loops ≥ 3	−0.20	✓ −0.00	✓ −0.00	Deep nesting penalty
EFFICIENCY TOTAL	max 1.0	0.8500	0.7000	Clamped [0.0, 1.0]
Contribution to Final	$\times 0.10$	0.0850	0.0700	Efficiency $\times 0.10$

Table 8: Algorithmic Efficiency via Static Code Analysis: ≥ 2 Nested Loops = Penalty

Final Weighted Score Summary (Round 4)				
Metric	Weight	Gpt-5.2-pro	gpt-oss-120b	Weighted Contribution
Correctness	45%	1.0000	0.0000	0.4500 vs. 0.0000
Methodology	30%	1.0000	0.0000	0.3000 vs. 0.0000
Code Quality	15%	9.50/10	9.00/10	0.1425 vs. 0.1350
Alg. Efficiency	10%	0.8500	0.7000	0.0850 vs. 0.0700
Final Calculation				
$\sum$ Weighted Scores	100%	0.9775	0.2050	—
Score Differential	—	+0.7725 (Gpt-5.2-pro advantage)		77.25% margin
WINNER	—	Gpt-5.2-pro (Code_A Submission)		

Table 9: Weighted Score Aggregation Sums Prior Tables' Metrics

4.5 Round 4 Final Results

The Kimi K2 Thinking purple bubble in Figure 4 indicates the largest round improvement, but Gemini 2.5 Pro API, DeepSeek R1 05/28, and GPT-5.2-pro all started with higher initial scores. GPT-5.2-pro won the most rounds, represented by the largest bubble size. Figure 5 illustrates score progression throughout the full 4 rounds, with GPT-5.2-pro improving after the first iteration and maintaining its lead throughout the tournament. OpenAI gpt-oss-120b achieved the highest first round score at 0.9400, which was maintained for two additional rounds, but its code failed to execute in the final round - represented by the late orange line decline. Gemini 2.5 Pro API also improved after the first round to 0.9325, maintained its score, but was eliminated by Gpt-5.2-pro in the semifinal round. All 16 of the tournament seeds are shown in the Figure 6 statistical summary, indicating 2 models with scores of 0.90 or higher between proprietary and open-source LLMs for the 8 first round winners. For those winners, the number of ≥ 0.90 second round scores increased to 6 based on the same FDA FAERS task. Performance was primarily constant in the semifinals and finals, although a single LLM code failed to execute in both rounds.

The Figure 7 final championship bracket shows contestants, competitors, and scores for each of the tournament’s 15 matches. Top proprietary LLM Gpt-5.2-pro defeated the best open-source LLM OpenAI gpt-oss-120b in the final round by a score of 0.9775 to 0.2050. Gpt-5.2-pro saw an increase in its first round score from 0.8238 to 0.9775, maintaining that score throughout the tournament. gpt-oss-120b accomplished the top Round 1 score of all models at 0.9400, maintaining its score until the final round, with its code failing to execute; reflected by a score of 0.2050, and finishing runner-up. Kimi K2 Thinking saw the largest single round score improvement from 0.5350 to 0.9400, and Gemini 2.5 Pro API attained a small performance increase over the newer Gemini 3 Pro API in the first round of 0.7756 versus 0.7681, while Anthropic models struggled in the first two rounds, with scores not surpassing 0.4850 - in supplementary Round_1 model thoughts describing a lack of focus on the perfect score goal.

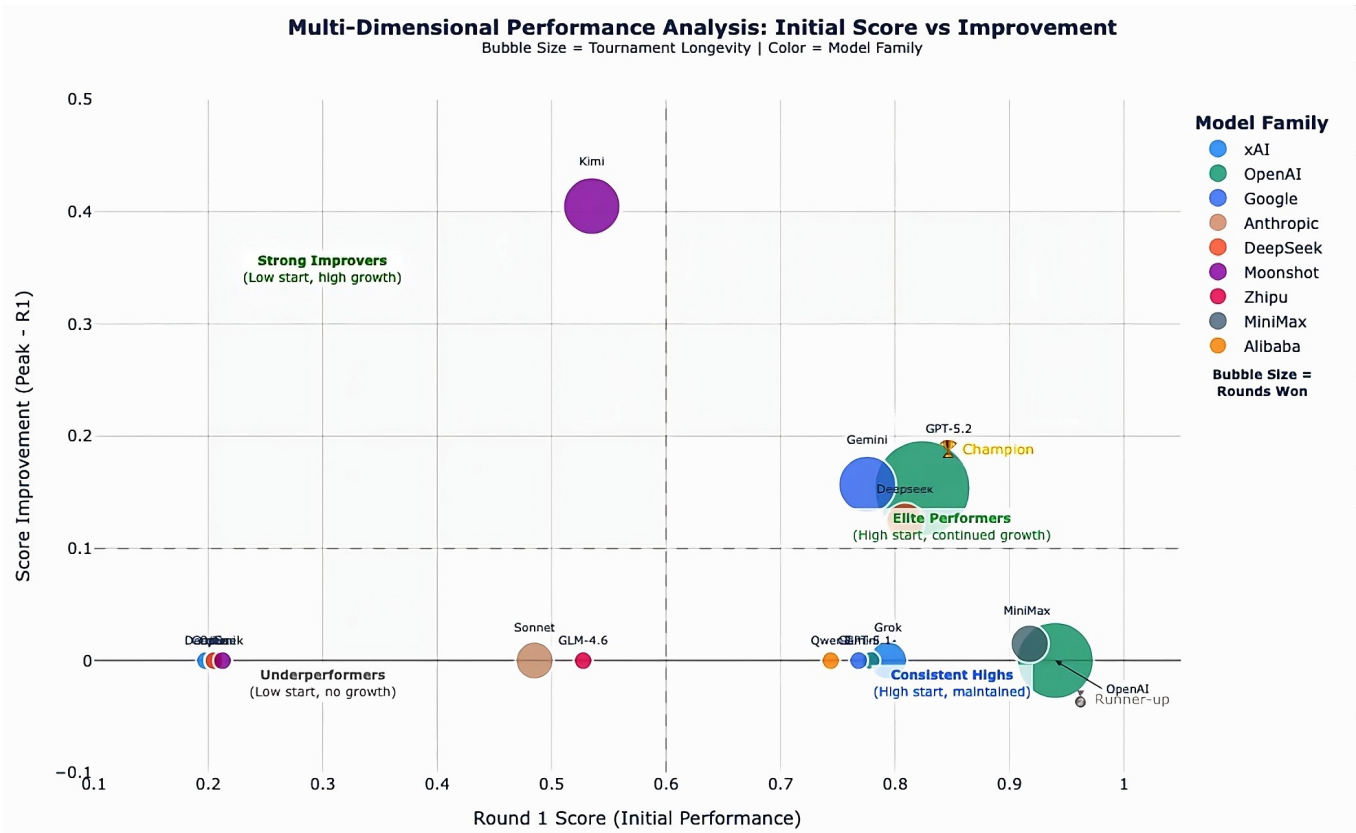


Figure 4: Largest Model Improvements After First Round are Highest

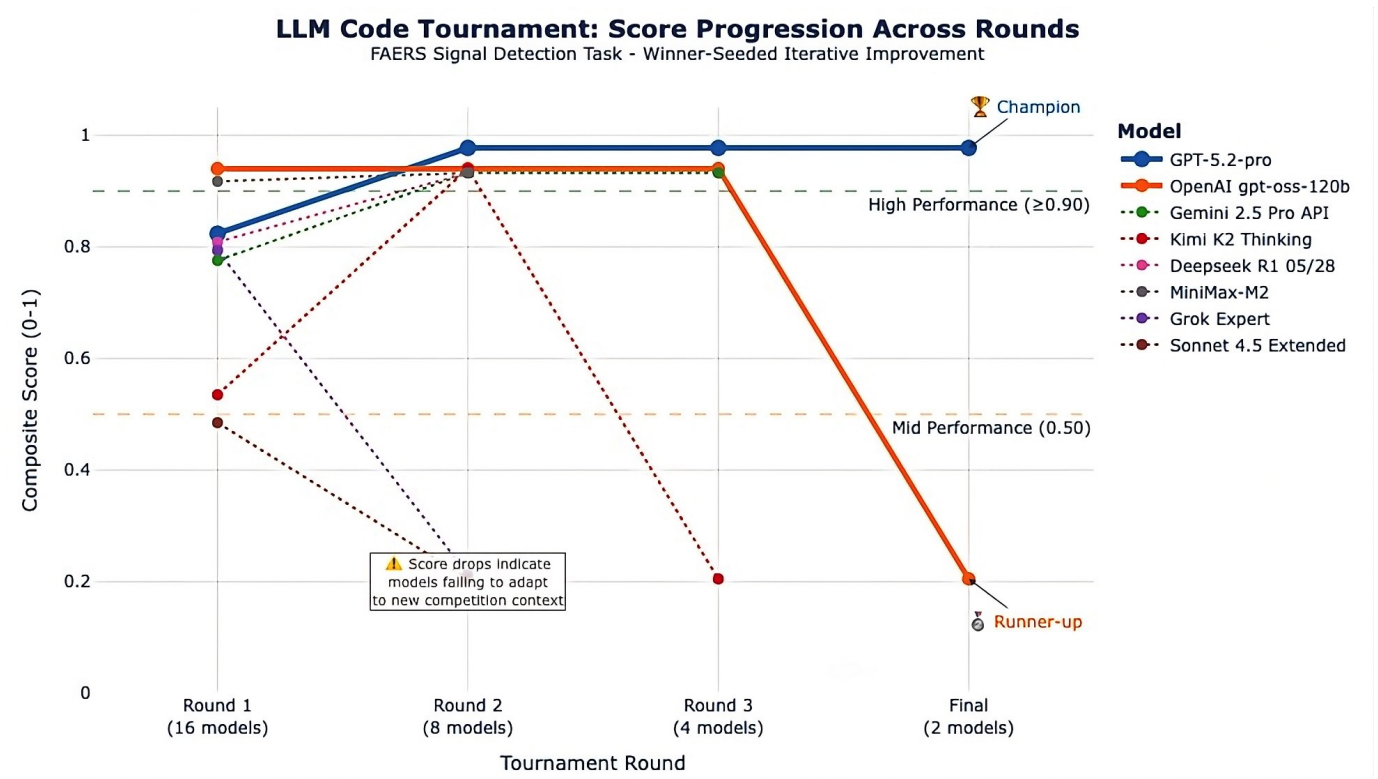


Figure 5: Scoring Progression of LLM Code: Inclines Represent Increases

Tournament Results: Complete Statistical Summary

● Proprietary LLMs | ● Open-Source LLMs | Color Coding: Green (≥ 0.90) | Blue (≥ 0.70) | Yellow (≥ 0.40) | Red (< 0.40)

Seed	Model Name	Type	Round 1	Quarter-finals	Semi-finals	Final	Mean	Trend	Result
04	Gpt-5.2-pro	Proprietary	0.8238	0.9775	0.9775	0.9775	0.9391	↗→	🏆
15	OpenAI gpt-oss-120b	Open-Source	0.9400	0.9400	0.9400	0.2050	0.7562	→↘	🥈
06	Gemini 2.5 Pro API	Proprietary	0.7756	0.9325	0.9325	—	0.8802	↗→	SF
11	Kimi K2 Thinking	Open-Source	0.5350	0.9400	0.2050	—	0.5600	↗↘	SF
02	Grok Expert	Proprietary	0.7938	0.2125	—	—	0.5031	↘	QF
08	Sonnet 4.5 Extended	Proprietary	0.4850	0.2050	—	—	0.3450	↘	QF
10	Deepseek R1 05/28	Open-Source	0.8088	0.9325	—	—	0.8706	↗	QF
14	MiniMax-M2	Open-Source	0.9175	0.9325	—	—	0.9250	→	QF
01	Grok 4.1	Proprietary	0.1975	—	—	—	0.1975	•	R1
03	Gpt-5.1-codex-max	Proprietary	0.7788	—	—	—	0.7788	•	R1
05	Gemini 3 Pro API	Proprietary	0.7681	—	—	—	0.7681	•	R1
07	Opus 4.5 Extended	Proprietary	0.2050	—	—	—	0.2050	•	R1
09	DeepSeek-V3.2	Open-Source	0.2050	—	—	—	0.2050	•	R1
12	Kimi K2 Instruct	Open-Source	0.2125	—	—	—	0.2125	•	R1
13	GLM-4.6	Open-Source	0.5275	—	—	—	0.5275	•	R1
16	Qwen3 Coder 480B	Open-Source	0.7438	—	—	—	0.7438	•	R1

Bracket Summary: Proprietary R1 Mean: 0.6034 | Open-Source R1 Mean: 0.6113 | Δ : +0.0078

Legend: ↗ Score Increased | → Score Stable | ↘ Score Decreased | • Single Round Only

Result Codes: 🏆 Champion | 🥈 Runner-Up | SF = Semifinalist | QF = Quarterfinalist | R1 = Round 1 Elimination

Figure 6: Statistical Scoring History for All 16 Models Across 4 Rounds

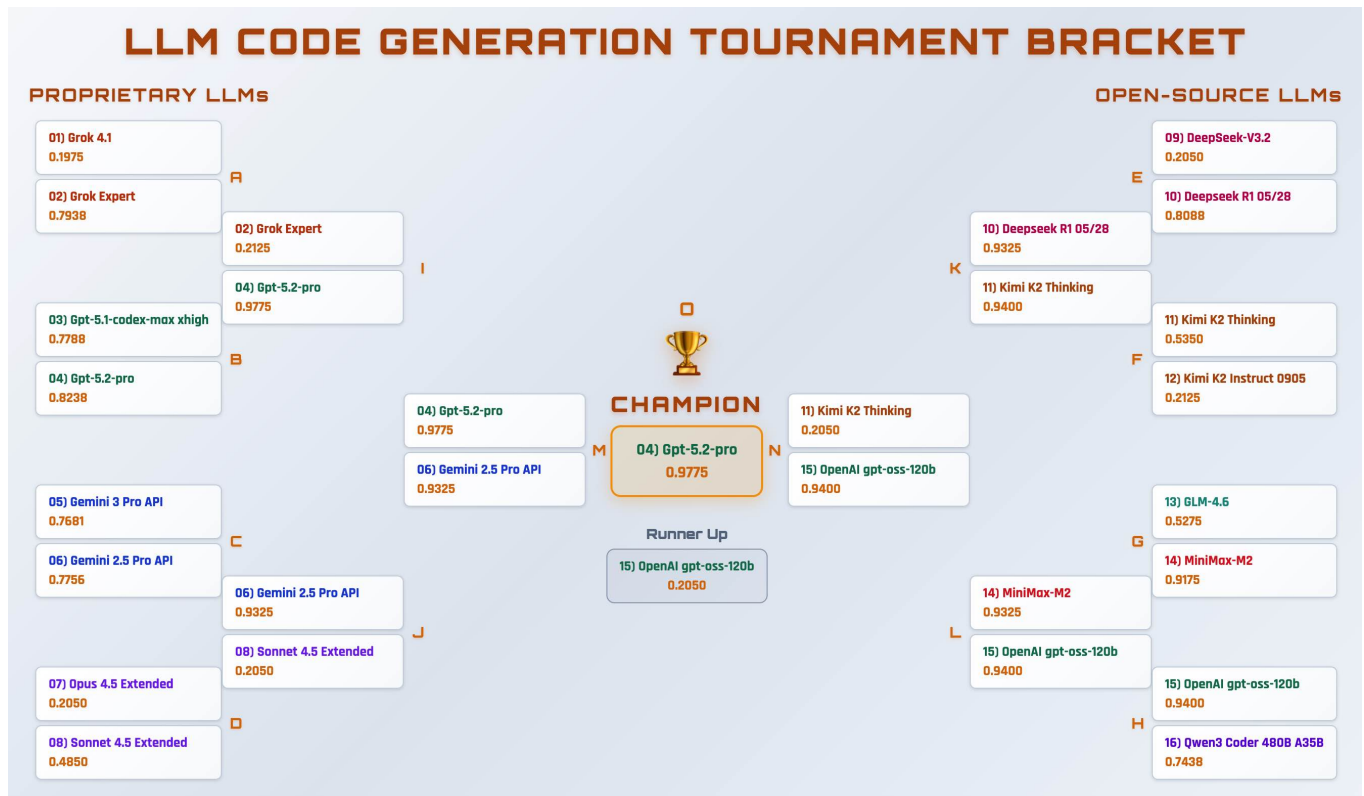


Figure 7: Final Championship Results of All 16 LLM Contestants and Scores

5 Discussion

5.1 Academic Paper References

Academic Paper References for Iterative LLM Code Improvement				
#	Paper Title	Authors	Year	Relevance to Competition
1	Self-Refine: Iterative Refinement with Self-Feedback [49]	Madaan et al.	2023	Core paradigm: Round N notebook → Round N+1 code
2	Evaluating Large Language Models Trained on Code [50]	Chen et al.	2021	Correctness metric foundation (pass@k)
3	Rethinking the Role of Demonstrations: What Makes In-Context Learning Work? [51]	Min et al.	2022	Prior Code_A/Code_B as few-shot examples
4	ChatDev: Communicative Agents for Software Development [52]	Qian et al.	2023	Multi-agent head-to-head code competition
5	CodeRL: Mastering Code Generation through Deep Reinforcement Learning [53]	Le et al.	2022	Execution-based reward (Correctness=45% weight)
6	CodeClash: Benchmarking Goal-Oriented Software Engineering [54]	Yang et al.	2025	Multi-round tournament to build the best Python function

Table 10: Academic Papers Relevant to Multi-Round Iterative Learning Process

The current competition builds off of techniques established in prior LLM application papers. For instance, the current study implements Self-Refine (Paper 1) across 4 rounds [49], where each model receives its prior round’s notebook as in-context learning demonstrations (Paper 3) [51]. Correctness scoring against a Reference Solution parallels CodeRL’s execution feedback (Paper 5) [53]. The head-to-head format is akin to ChatDev’s multi-agent collaboration (Paper 4) [52]. Additionally, the CodeClash goal-based benchmark from the SWE-bench Team regarding LLMs competing in multi-round game tournaments to build the best codebase for achieving a competitive objective was a framework for the study (Paper 6) [54]; where an increased competition pool size from 8 to 16 and a medical objective were additive. The FDA FAERS data files [48] used in the competition task are known in literature to be effective for providing safety assessments throughout the life cycle of a drug [55], providing new and unexpected signals of adverse drug reactions [56], and discovering top drugs based on events leading to death [57].

5.2 Difficulty Benchmarking

Answer key creation was easier for the notebook to establish than executing either Code A or Code B due to the solution being fully deterministic with no ambiguity, while competition code must consider many more rows and drug-reaction combinations. Difficulty standards of the Table 11-Table 15 were calculated by Opus 4.5 Extended.

Reference Solution Code

- No programmatic creativity requirements
- Already knows how the tables must be merged
- Already knows correct logic, date, and quarter parsing rules

Competition Code Must Run

- The full sampled FAERS subset, which can be thousands of rows
- All combinations of drug × reaction considered, yielding very large numbers
- All time-cumulative quarter calculations, which multiplies the number of operations

5.3 Difficulty Benchmarking Standards

Software Engineering Complexity Standards				
Standard	Metric	Task 1 Value	Industry Benchmark	Difficulty Rating
McCabe	Cyclomatic Complexity	25–35	>20 = High Risk	8.0/10
Halstead	Difficulty (D)	~32	>30 = Challenging	7.5/10
Halstead	Effort (E)	~90,000	>50K = Substantial	8.0/10
Halstead	Time to Program	~83 min	>60 min = Complex	7.0/10
COCOMO II	Effort	0.6 person-months	0.5–1.0 = Medium	6.5/10
Lines of Code	Implementation Size	150–230 LOC	100–300 = Medium	6.0/10
			Subtotal Average	7.17/10

Table 11: Software Engineering Complexity Assessment Metrics Derived from Static Analysis of detect_signal_emergence_improved Function Specification (TASK_1_SPEC)

AI/ML Benchmark Alignment Standards			
Benchmark	Criterion	Task 1 Mapping	Score (0–10)
HumanEval	Function Signature	Exact DataFrame params required	7.0
HumanEval	Docstring Comprehension	TASK_1_SPEC interpretation	8.0
HumanEval	Edge Case Handling	Empty inputs, invalid dates, missing cols	9.0
HumanEval	Algorithmic Correctness	PRR formula + temporal emergence	9.0
HumanEval	Output Validation	6-column schema compliance	6.0
MBPP	Problem Category	Level 4–5 (Advanced/Expert)	8.5
BigCodeBench	Tier Classification	Tier 3–4 (Domain + Multi-source)	8.0
APPS	Difficulty Band	Interview-level (comparable)	7.5
		Subtotal Average	7.88/10

Table 12: Established AI/ML Code Generation Benchmarks. Edge Case Handling Scored Highest (9.0)

FDA/Pharmacovigilance Regulatory Standards			
Standard	Component	Implementation Requirement	Complexity Multiplier
ICH E2B(R3)	Case Identification	primaryid linkage (3 tables)	1.5×
ICH E2B(R3)	Drug Characterization	role_cod suspect filtering	1.2×
ICH E2B(R3)	Reaction Coding	MedDRA PT handling	1.1×
ICH E2B(R3)	Temporal Analysis	event_dt → quarter conversion	1.8×
CIOMS VI	Signal Detection	PRR ≥ 2.0, N ≥ 3 threshold	1.3×
EMA Guidelines	Disproportionality	Expected vs. observed ratio	1.4×
		Cumulative Complexity Factor	2.97×
Base Difficulty	5.0 (standard data task)	5.0	—
Adjusted Difficulty	$5.0 \times 2.97 \div 2$	Derived Difficulty Score	7.43/10

Table 13: FDA Complexity: Expected = (drug_cases * reac_cases) / total_cases; prr = pair_cases / expected

Data Complexity Assessment (FAERS-Specific)			
Data Characteristic	Description	Challenge Level	Score (0–10)
Relational Structure	3-table inner joins required	High	8.0
Date Format Variability	YYYYMMDD, YYYYMM, decimal suffixes	High	8.5
Missing Data Prevalence	Requires dropna/fillna handling	Moderate	6.5
Case Deduplication	nunique() on primaryid	Moderate	6.0
Volume Scalability	400K+ records (full dataset)	High	7.5
Schema Variability	Column casing, whitespace	Moderate	6.5
Temporal Granularity	Quarterly aggregation logic	High	8.0
		Subtotal Average	7.29/10

Table 14: FAERS Dataset Complexity: Schema Variability (6.5) Applied via columns.str.lower().str.strip()

Composite Difficulty Score Calculation				
Dimension	Source Table	Score	Weight	Weighted Score
Software Engineering	Table A	7.17	20%	1.434
AI/ML Benchmarks	Table B	7.88	25%	1.970
Regulatory Standards	Table C	7.43	25%	1.858
Data Complexity	Table D	7.29	20%	1.458
Domain Expertise Required	(Expert estimate)	8.50	10%	0.850
TOTAL			100%	7.57/10

Table 15: Weighted Composite Difficulty Score Representing Task-Level Complexity Across 4 Tournament Rounds

6 Limitations and Future Work

All LLMs run in the study were either on \$20 OpenAI, \$20 Anthropic, \$30 xAI monthly plans, or using pay per token API playgrounds: Gpt \$7.47, Open-source \$0.59; \$78.06 Total. A limitation was in accessing additional high tier models including Google AI Ultra's "Deep Think" feature "which uses parallel processing to evaluate multiple options and refine answers quickly. This results in fewer errors in complex tasks like advanced math, scientific modeling, and high-stakes financial projections". This option was cost prohibitive for the study at a standard price of \$249.99/month [58]. Similarly, xAI's SuperGrok Heavy is meant "for professionals performing heavy reasoning, automation workflows, dense analytical tasks, coding support, multi-step problem solving" [59], but costs \$300/month. In either of these two model tiers, the manufacturer's current models likely evaluate multiple generations based on the same prompt to increase the likelihood of returning a single best answer.

Another limitation was in the development of the Round 0 notebook used by all 16 models to generate their first code submission that was run in the Round 1 competition notebook. The Round 0 notebook was based on lower score initial competitors (0.8750/0.8875) which may have limited Round 1 score performance. Similarly, the Round 0 notebook used to generate Round 1 code was based on a different correctness metric regarding a reference standard, which may have also caused lower than expected Round 1 scores. Regardless, the goal provided to both Round 1 and Round 2-4 contestants remained the same "to achieve a perfect score of 1.0" despite minor notebook differences. Attempts to increase FAERS Dataset incorporation beyond the tournament's "0.1" sample fraction setting were not successful, as shown in Prompt_10. The notebook development process did occur a truncation in the number of notebook outputs from the Round 0 to Round 1, as the competition task became better defined. This main effect was that the AI peer review process instructions originally outputted inside the notebook containing competition functions, scores, and additional insights became reduced, and were relocated as file outputs.

To accommodate for the truncation, code generations for Rounds 2-4 were provided the "Instructions for Improvement" section from Round 0 to assist LLMs in the competition task, and the exact function response format. Based on the CodeClash paper's findings [54] regarding limitations of processing end-of-match log files, a similar approach was taken in the current study by solely focusing on the processing of a match winner's previously completed notebook. Suitable information existed regarding both competitor's code, code results, and other adjustments based on the notebook's format and system to make informed code changes. The additional +100 line "08_competition_report.json" would likely overwhelm LLMs and cause poor final scores, based on the author's best prior results limiting the number of inputs for working and high performance code [60, 61, 62]. Another limitation was observed in high correctness and methodology score penalties for unexecutable competition code (-0.7500), as there were 7 scores at 0.2125 or below. Future studies will focus on oncology LLM-generated code at scale.

7 Conclusions

Human code iterations have increased efficiency with the aid of interactive debugging and profiling tools. However, manual optimization, time-consuming, and labor-intensive limitations have led to worker fatigue and diminishing returns. AI code iterations have automated the use of algorithms with the use of search-based optimization, genetic programming, automated program repair, reinforcement learning, and ML-guided compilers. This includes DeepMind's 2022 AlphaTensor [9] and 2023 AlphaDev [11] projects that used reinforcement learning to discover new matrix multiplication algorithms and algorithm discovery being treated as a game. However, these type of pre-LLM coding iterations were typically rigid - in that code must be machine-readable, with the improvement goal encoded as a fitness function or reward. Fast forward to late 2025, and LLMs now consistently provide an enhanced number of utilities to quickly iterate code and utilize natural language interactions with humans across full R&D areas.

The 2025 DeepMind AlphaEvolve [17] now uses algorithmic AI agents combining evolutionary search with LLM-guided code generation to iteratively evolve algorithms. Several academic papers have laid the groundwork regarding iterative refinement [49], LLM evaluation [50], in-context learning [51], chat-based coding competitions [52], and reward-based code generation [53]. The November 2025 CodeClash LLM-generated code competition by Yang et al. [54] provided a compelling 8 contestant multi-game format in which Claude Sonnet 4.5 took the highest overall score. Since the results were published, November/December 2025 industry additions of Grok 4.1, Gpt-5.2-pro, Gemini 3 Pro, Opus 4.5, and DeepSeek-V3.2 were found to warrant a larger 16 contestant pool based on the pharmaceutical industry relevant FDA Adverse Event Reporting System (FAERS) latest quarterly data files (25Q3). The 8 proprietary vs. 8 open-source LLM single-elimination code generation tournament was based on a Python competition notebook developed by Opus 4.5 Extended. A total of 12 software engineering prompts were utilized in order to 1) Optimize the FAERS four signal reference-solution output based on the "First calendar quarters where pair becomes a safety 'signal' (PRR $\geq$ threshold AND count $\geq$ minimum cases)", 2) Fix errors, and 3) Develop the prompt instructing LLMs to generate competition code.

First Round winners saw upsets between proprietary LLM manufacturer pairs of Gemini 2.5 Pro API (0.7756) defeating Gemini 3 Pro API (0.7681); as well as open-source LLM manufacturer pairs of DeepSeek R1 05/28 (0.8088) defeating DeepSeek-V3.2 (0.2050). The surprising first round best score was achieved by OpenAI gpt-oss-120b at 0.9400. To generate subsequent rounds of competition code, each winning LLM was provided with the competition prompt and their prior notebook based on competitors' code, match code, and match results to iteratively learn and generate new code. Round 1-2 iterative learning scores increased for 4/8 of the contestants, most notably with Kimi K2 Thinking having the largest single round increase of the tournament from 0.5350 to 0.9400 (+0.405). From Round 2 to Round 3, 3/4 teams maintained their previous round's score with semifinal proprietary LLM scores featuring a Gpt-5.2-pro (0.9775) victory over Gemini 2.5 Pro API (0.9325). Open-source LLM scores of OpenAI gpt-oss-120b (0.9400) overcame Kimi K2 Thinking (0.2050). Championship results featured Gpt-5.2-pro maintaining its 0.9775 score victory over OpenAI gpt-oss-120b (0.2050); marking gpt-oss-120b's only non 0.9400 score. Thus, Gpt-5.2-pro emerged as an iterative learner with consistent scores and this Championship tournament winner.

8 Prompts

Prompts 01-06: Notebook Generation

Prompt_01: Initial Python FAERS Competition Notebook.

Prompt for Generating a Python Notebook: Head-to-Head LLM Code Competition Using FDA FAERS Pharmacovigilance Data Purpose and Context

Generate a Python notebook designed for execution on a Google Colab T4 GPU environment that implements a structured head-to-head competition framework for evaluating LLM-generated code. The competition uses real-world pharmacovigilance data from the FDA Adverse Event Reporting System (FAERS) Q3 2025 Quarterly Data Extract. [See Supplementary Notebook_Generation for Full Prompt]

Prompt_02: Improvements: Drive, Sections, ASCII Dataset Samples. Focus your resources on improving the ipynb. Make sure google drive is incorporated in order to upload the 7 txt file datasets, and also download the full results to specific directories (no data generation is required). Separate cells by section for the Python notebook appropriately. Include instructions for relevant sections that are adjustable. A sample of the dataset files is included for reference: ASCII Dataset Samples.txt. It is important for the competition to surround around the full dataset files (not included due to size). Remove the FDA link, and include this instead: FDA Adverse Event Reporting System (FAERS): Latest Quarterly Data Files. Remove the few shot learning feature inside the competition (only the LLMs' code will run head to head). Be sure to more clearly detail how exactly each LLM's code is incorporated. [ASCII Dataset Samples.txt]

Prompt_03a: ChatGPT Competition Task A, B, C Selection. For the original attached notebook: Describe which and how the 4 tasks were run for Code A and Code B (were each of the tasks run for both sets of code?). Provide 3 other more constructive and challenging tasks based on the 7 file FAERS dataset in the competition (that no other researchers have likely done before on this dataset)? Right now correctness is 100%, as this is likely for more trivial tasks. The other issue between Code A and Code B is that the runtimes are not consistent across multiple experiments based on the hardware (and hence the efficiency scores vary). What would be some ways to address this? Or should this issue resolve with more challenging and longer tasks? The overall scores for Code A and B should be consistent for each run and not be thrown off by fluctuating runtimes. [USER_FAERS_LLM_Competition.ipynb]

Prompt_03b: ChatGPT Competition Additional Task Selection. No. Similar format and length the originals. Like: Task C — Treatment Pathway Reconstruction From THER + INDI + OUTC Objective: Identify the sequence of drug therapies for a given indication and determine outcome likelihood based on therapy order. Requires: * Merge THER with DRUG and INDI * Reconstruct drug start/end dates per case * Infer therapy sequences * Model outcomes by treatment order (e.g., Drug A → Drug B vs. Drug B → Drug A) Novelty: * FAERS is not commonly used for pathway inference due to date quality * This becomes a deep reasoning task for LLMs Hard for LLMs: * Cleaning inconsistent dates * Sorting sequences * Joining three files with time logic * Aggregating outcome patterns [USER_FAERS_LLM_Competition.ipynb]

Prompt_03: Incorporation of Increased Competition Tasks Difficulty. Your goal is to focus only on updating the provided notebook with the attached more relevant competition tasks that are more challenging and will allow for larger differences in the "Correctness" metrics. The results must be reproducible every time the notebook is run, therefore the "Efficiency" metric should be modified from runtimes to an algorithmic complexity proxy, or other suitable method to approximate runtime. Code_A score and Code_B score must not change every time the notebook is re-run. Include runtimes, but not for the use of scoring. New Code_A and Code_B code simulating LLM-generated code must be included to address the four included replacement competition tasks. Keep the function all sections of the attached notebook. Return a single new Python notebook. "START NEW COMPETITION TASKS" [See Supplementary for Full Tasks]

Prompt_04a: ChatGPT Task A FDA Relevancy Identification. Based on this notebook. Which competition task is most relevant to FDA, and why? Detail in full. [USER_3rd_FAERS_LLM_Competition.ipynb]

Prompt_04: Error Fixes, Single Competition Task A Identification. Your goal is to focus only on providing a new Python notebook that fixes the attached notebook errors, and focuses only on the single "Task A: Cross-Table Temporal Signal Emergence Detection" competition task. Therefore there should only be one set of Code A and Code B competition code. Keep the function of all sections of the attached notebook. Return a single new Python notebook. [USER_3rd_FAERS_LLM_Competition.ipynb]

Prompt_05: Several Optimizations, Task A to Task 1 Rename. Produce one optimized Python notebook. Task A should be renamed Task 1. Additionally, all subsequent rounds will aim to improve on the same Task 1. The competition task code needs to be explicitly commented (it should be clear to the user how the task code functions, and where all it is incorporated into the notebook). If one or both of the competition Code do not run, the notebook should finish and state the results appropriately. In the Round Results output: "Algorithmic Efficiency" should be renamed "Algorithmic Effic." The Peer Review Process cell output needs to include results and code for both Code A and B in the same output. This Peer Review Process output will be provided to both the winner and loser of the competition. The Peer Review Process output should include all necessary instructions, so either LLM can make corrections based on the single Peer Review Process output. The instructions should specify LLMs to output their replacement Python functions in Python, not JSON. Keep the function of all sections of the attached notebook. Return a single new Python notebook. [USER_4th_FAERS_LLM_Competition_TaskA_Fixed.ipynb]

Prompt_06: Temporal-Signal Task Defined Outside Competition Code. It appears that the the temporal-signal task is implemented exclusively in the two LLM-generated functions. Should the task be defined in the notebook outside of the competition code? Will future rounds of code be deriving the task from the Code and not explicitly defined in the notebook? Is it possible to have the task defined separately in it's own cell?

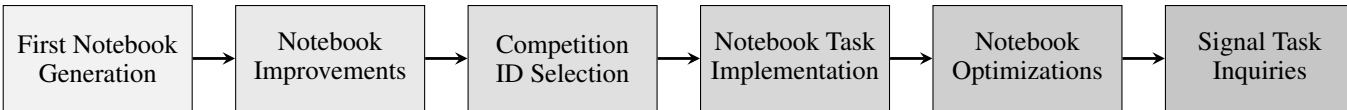


Table 16: Opus 4.5 Extended and ChatGPT, Brackets are Attachments. See Supplementary for Prompts, Thoughts, and Outputs

Prompts 07-12: LLM Code Gen & Notebook Generation

Prompt_07: Meta Prompt for Single New Code Competition Generation. Design a prompt that will be provided to a LLM that takes in a notebook such as that which is attached, and outputs a single new Code to compete in future notebook competitions (don't specify Code A or Code B; the generated code could be used for either case). It is important that your prompt directs the LLM to generate new competition code that is based on the notebook, and sections such as the PEER REVIEW PROMPT output. Essentially, the future LLM should be able to make informed decisions from the notebook to achieve a final score of "1.0" based on Correctness, Methodology, Code Quality, and Algorithmic Efficiency. Inform the LLM that it can execute code on its own if that is believed to improve the final score of the Code in the competition. Also instruct the LLM to specify which aspects of notebook sections were used in its decision making process to obtain its new CODE. [USER_6th_FAERS_LLM_Competition_Task1_v2.ipynb]

Prompt_08: New Notebook Outputs Exact Competition Task in Drive. Based on the attached notebook, return a new Python notebook that outputs in Drive the exact answer that was used for the task competition, and each of Code A and Code B answers that correspond to the exact solution to allow for head to head comparisons. Currently, there are three output files that don't appear to address these concerns (other similar information needed for a publication should also be outputted, such as (concise and general explanations regarding how the competition task solution and Code A and Code B answers were formed independent on Code function). Additionally, one of the notebook cells outputs several prior files that are all in the same directory (Files in Results Directory (/content/drive/MyDrive/Colab Notebooks/Inputs/FAERS_LLM_Competition/results:)). Instead, for each new time the notebook is run: a new time stamped directory showing only the output files for that run should be displayed in the current notebook (including any other output files that show the solution, Code A and Code B answers, and other relevant files needed for publication). Keep the functional aspects of all sections of the attached notebook. Return a single new Python notebook. [Tournament_FAERS.ipynb]

Prompt_09: Code Correctness Validation Accuracy Inquiries. For the round results, do the following csv files make sense? Are these what are only being considered for the Correctness metric? Are the 02_Code_A_output entries prior to its last four entries prior guesses? 03_Code_B is the same as 01_reference but didn't receive a perfect Correctness score. Be concise and specific.

```
===== ROUND RESULTS (Deterministic
Scoring) ===== Code_A Score: 0.8675
Correctness: 0.9500 (x0.45) Methodology: 0.9000 (x0.30) Code Quality: 8.00/10 (x0.15) Algorithmic Effic.: 0.5000 (x0.10) Code_B
Score: 0.8875 Correctness: 0.9000 (x0.45) Methodology: 0.9000 (x0.30) Code Quality: 8.50/10 (x0.15) Algorithmic Effic.: 0.8500
(x0.10)
```

Prompt_10: Output File Validations to Other Output Files. (SAMPLE_FRACTION Fixed at 0.1) Make sure that the most recent attached notebook and the files it outputs correspond to each other, as mentioned in this most recent output in this conversation. This functionality should exist for any SAMPLE_FRACTION size. Checks of csv files should correlate between how close Code A and Code B is to the reference solution, with consistency being of publishable quality. Keep the functional aspects of all sections of the attached notebook. Return a single new Python notebook. [USER_Tournament_FAERS_Enhanced.ipynb]

Prompt_11: Error Fixes, Maintaining Notebook Functionality. Fix errors. Keep the functional aspects of all sections of the attached notebook. Return a single new Python notebook. [USER_Fix_Tournament_FAERS_v2.ipynb]

Prompt_12: Code Methodology Based on Code Submissions Redefined. Remove CODE_A_METHODODOLOGY and CODE_B_METHODODOLOGY and their corresponding text and any impact they have on the attached notebook. Methodology result score should only be based on code practices and/or execution success. Keep the four Correctness criteria and three other results metrics in Section 6.1, but focus more on Code A and Code B Total Scores in subsequent sections. Keep the functional aspects of all sections of the attached notebook. Return a single new Python notebook. [A_RD1_Tournament_FAERS.ipynb]



Table 17: Opus 4.5 Extended, Brackets are Attachments. See Supplementary for Prompts, Thoughts, and Outputs

Prompts: Rounds 1-4 Competition Code Generation

Round_1_Prompt: You are an expert Python peer reviewer and developer specializing in pharmacovigilance data analysis. Your task is to analyze the attached competition notebook and generate a single, optimized Python function that aims to achieve a perfect score of 1.0 across all evaluation metrics (Correctness, Methodology, Code Quality, and Algorithmic Efficiency). The code you generate can be used as either Code_A or Code_B in a future head-to-head LLM code competition. Utilize the existing notebook's code, results, peer review output, etc. It is important that your new Python function addresses the task exactly as specified in TASK_1_SPEC, and follows the def detect_signal_emergence_improved response format in Section 7.3. In the "Improvements made" section of the response format, provide a commented numerical list using three numbers (1., 2., 3.) to describe the steps you took to reach your single Python function. If you are able, execute your code to ensure functionality and performance against the existing Code A and B. No modifications to the notebook's installs and imports can be made. Return only your new Python function and the concise synopsis.

[USER_6th_FAERS_LLM_Competition_Task1_v2.ipynb] OR [JSON_USER_6th_FAERS_LLM_Competition_Task1_v2.ipynb]

Multi_Round_Prompt: You are an expert Python peer reviewer and developer specializing in pharmacovigilance data analysis. Your task is to analyze the attached competition notebook and generate a single, optimized Python function that aims to achieve a perfect score of 1.0 across all evaluation metrics (Correctness, Methodology, Code Quality, and Algorithmic Efficiency). The code you generate can be used as either Code_A or Code_B in a future head-to-head LLM code competition. Utilize the existing notebook's code, results, INSTRUCTIONS FOR IMPROVEMENT below, etc. It is important that your new Python function addresses the task exactly as specified in TASK_1_SPEC, and follows the def detect_signal_emergence_improved response format. In the "Improvements made" section of the response format, provide a commented numerical list using three numbers (1., 2., 3.) to describe the steps you took to reach your single Python function. If you are able, execute your code to ensure functionality and performance against the existing Code A and B. No modifications to the notebook's installs and imports can be made. Return only your new Python function and the concise synopsis.

"START INSTRUCTIONS FOR IMPROVEMENT"

===== INSTRUCTIONS FOR IMPROVEMENT =====

Your task is to create an IMPROVED implementation of Task 1.

IMPORTANT: The task specification (TASK_1_SPEC) is FIXED - do not change the requirements. Only improve HOW you implement the solution.

1. ANALYZE BOTH SUBMISSIONS:

- Compare approaches used by Code_A and Code_B
- Identify techniques that led to higher scores
- Note any errors that caused failures

2. IDENTIFY IMPROVEMENTS (by score impact):

- Correctness (45%): Scored against the Reference Solution
- Methodology (30%): Error-free execution, proper data handling
- Code Quality (15%): Docstrings, comments, functions, type hints
- Algorithmic Effic. (10%): Vectorization, groupby, avoid nested loops

3. OUTPUT YOUR IMPROVED SOLUTION IN PYTHON:

Provide your complete replacement function as Python code (not JSON).

Your function MUST:

- Accept: demo_df, drug_df, reac_df, min_cases=3, prr_threshold=2.0
- Return: DataFrame with columns: ['drug_name', 'reaction', 'emergence_quarter', 'emergence_prr', 'total_cases', 'is_signal']
- Set result variable: result = your_function(demo_df, drug_df, reac_df)

RESPONSE FORMAT - Output Python code like this:

```
"""python
def detect_signal_emergence_improved(demo_df, drug_df, reac_df, min_cases=3, prr_threshold=2.0):
    """
```

Improved implementation of Task 1 (TASK_1_SPEC).

Improvements made:

- [List your improvements here]

```
"""
```

Your improved implementation

...

```
return result_df
```

REQUIRED: Set result variable

```
result = detect_signal_emergence_improved(demo_df, drug_df, reac_df)
```

```
print(f"Found len(result) signals")
```

"START COMPETITION NOTEBOOK" [Z_1RD_Tournament_FAERS.ipynb], $Z \in \{A, \dots, N\}$

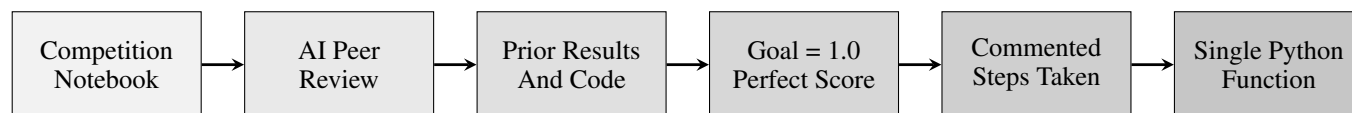


Table 18: Opus 4.5 Extended, Brackets are Attachments. See Supplementary for Prompts, Thoughts, and Outputs

9 Data Availability, Zenodo [63]

LaTeX Source Code

1. Latex Code Source, 11 Files

Notebook Generation Inputs

2. Notebook_Generation, 2 Files
3. Input_02, 1 File
4. Input_03a, 1 File
5. Input_03b, 1 File
6. Input_03, 1 File
7. Input_04a, 1 File
8. Input_04, 1 File
9. Input_05, 1 File
10. Input_07, 1 File
11. Input_08, 1 File
12. Input_09, 3 Files
13. Input_10, 1 File
14. Input_11, 1 File
15. Input_12, 1 File

Notebook Generation Outputs

16. Output_01, 5 Files
17. Output_02, 8 Files
18. Output_03, 2 Files
19. Output_04, 3 Files
20. Output_05, 2 Files
21. Output_06, 4 Files
22. Output_07, 11 Files
23. Output_08, 1 File
24. Output_09, 2 Files
25. Output_10, 2 Files
26. Output_11, 12 Files
27. Output_12, 2 Files

Tournament Bracket Rounds

Competition Notebooks

28. Round 0 Notebook, 1 File
29. Round 1 Notebook, 1 File
30. Round 2 Notebook, 1 File
31. Round 3 Notebook, 1 File
32. Round 4 Notebook, 1 File

Round 1 Bracket Outputs

33. Round_1, 2 Files
34. A, 10 Files
35. B, 21 Files
36. C, 10 Files
37. D, 9 Files
38. E, 9 Files
39. F, 9 Files
40. G, 10 Files
41. H, 10 Files

Round 2 Bracket Outputs

42. Round_2, 2 Files
43. I, 29 Files
44. J, 29 Files
45. K, 32 Files
46. L, 32 Files

Round 3 Bracket Outputs

47. Round_3, 2 Files
48. M, 12 Files
49. N, 11 Files

Round 4 Bracket Outputs

50. Round_4, 2 Files
51. O, 11 Files

10 References

- [1] Python.org. This page is devoted to various tips and tricks that help improve the performance of your python programs. URL: <https://wiki.python.org/moin/PythonSpeed/PerformanceTips>.
- [2] Nachiappan Nagappan, E. Michael Maximilien, Thirumalesh Bhat, and Laurie Williams. Realizing quality improvement through test driven development: results and experiences of four industrial teams. *Empirical Software Engineering*, 13(3):289–302, 2008. doi:10.1007/s10664-008-9062-z.
- [3] Andreas Antoniou and Wu-Sheng Lu. *Practical Optimization: Algorithms and Engineering Applications*. Texts in Computer Science. Springer US, New York, NY, 2021. URL: <https://link.springer.com/10.1007/978-1-0716-0843-2>, doi:10.1007/978-1-0716-0843-2.
- [4] Python.org. The official home of the python programming language. URL: <https://www.python.org/downloads/release/python-3110/>.
- [5] DX. Learn how dora metrics help engineering teams boost delivery speed, quality, and reliability—plus how ai is reshaping software performance measurement. URL: <https://getdx.com/blog/dora-metrics/>.
- [6] Microsoft Developer. Boost developer productivity with expert insights on developer experience. learn how software developers can optimize workflow and improve code quality. URL: <https://developer.microsoft.com/en-us/developer-experience>.
- [7] smartbear.com. A successful peer review strategy requires balance between strictly documented processes and a non-threatening, collaborative environment. URL: <https://smartbear.com/en/learn/code-review/best-practices-for-peer-code-review/>.

- [8] bbc.com. Google deepmind's alphadev discovers a more efficient sorting algorithm, improving computing speed and energy use globally. URL: <https://www.bbc.com/news/articles/c9wd141pd5lo>.
- [9] Google DeepMind. In our paper, published today in nature, we introduce alphasense, the first artificial intelligence (ai) system for discovering novel, efficient, and provably correct algorithms for fundamental task. . . , December 2021. URL: <https://deepmind.google/blog/discovering-novel-algorithms-with-alphasense/>.
- [10] Alhussein Fawzi, Matej Balog, Aja Huang, Thomas Hubert, Bernardino Romera-Paredes, Mohammadamin Barekatain, Alexander Novikov, Francisco J. R. Ruiz, Julian Schrittwieser, Grzegorz Swirszcz, David Silver, Demis Hassabis, and Pushmeet Kohli. Discovering faster matrix multiplication algorithms with reinforcement learning. *Nature*, 610(7930):47–53, October 2022. URL: <https://www.nature.com/articles/s41586-022-05172-4>, doi:10.1038/s41586-022-05172-4.
- [11] Google DeepMind. In our paper published today in nature, we introduce alphadev, an artificial intelligence (ai) system that uses reinforcement learning to discover enhanced computer science algorithms – surpassing th. . . , October 2022. URL: <https://deepmind.google/blog/alphadev-discovers-faster-sorting-algorithms/>.
- [12] Daniel J. Mankowitz, Andrea Michi, Anton Zhernov, Marco Gelmi, Marco Selvi, Cosmin Paduraru, Edouard Leurent, Shariq Iqbal, Jean-Baptiste Lespiau, Alex Ahern, Thomas Köppe, Kevin Millikin, Stephen Gaffney, Sophie Elster, Jackson Broshear, Chris Gamble, Kieran Milan, Robert Tung, Minjia Hwang, Taylan Cemgil, Mohammadamin Barekatain, Yujia Li, Amol Mandhane, Thomas Hubert, Julian Schrittwieser, Demis Hassabis, Pushmeet Kohli, Martin Riedmiller, Oriol Vinyals, and David Silver. Faster sorting algorithms discovered using deep reinforcement learning. *Nature*, 618(7964):257–263, 2023. URL: <https://www.nature.com/articles/s41586-023-06004-9>, doi:10.1038/s41586-023-06004-9.
- [13] William B. Langdon and Mark Harman. Optimizing existing software with genetic programming. *IEEE Transactions on Evolutionary Computation*, 19(1):118–135, February 2015. URL: <https://ieeexplore.ieee.org/abstract/document/6733370>, doi:10.1109/TEVC.2013.2281544.
- [14] QUANTNET. Faster sorting algorithms discovered using deep reinforcement learning. URL: <https://quantnet.com/threads/faster-sorting-algorithms-discovered-using-deep-reinforcement-learning.54312/>.
- [15] GitHub. Join the world's most widely adopted, ai-powered developer platform where millions of developers, businesses, and the largest open source community build software that advances humanity., 2025. URL: <https://github.com/>.
- [16] Stack Overflow. Stack overflow: Newest questions, 2025. URL: <https://stackoverflow.com/questions>.
- [17] DeepMind. New ai agent evolves algorithms for math and practical applications in computing by combining the creativity of large language models with automated evaluators. URL: <https://deepmind.google/blog/alphaevolve-a-gemini-powered-coding-agent-for-designing-advanced-algorithms/>.
- [18] Alexander Novikov, Ngan Vũ, Marvin Eisenberger, Emilien Dupont, Po-Sen Huang, Adam Zsolt Wagner, Sergey Shirobokov, Borislav Kozlovskii, Francisco J. R. Ruiz, Abbas Mehrabian, M. Pawan Kumar, Abigail See, Swarat Chaudhuri, George Holland, Alex Davies, Sebastian Nowozin, Pushmeet Kohli, and Matej Balog. Alphaevolve: A coding agent for scientific and algorithmic discovery. *arXiv*, (arXiv:2506.13131), 2025. arXiv:2506.13131. URL: <http://arxiv.org/abs/2506.13131>, doi:10.48550/arXiv.2506.13131.
- [19] SWEbench. URL: <https://www.swebench.com/>.
- [20] Digital Applied. Llm comparison guide: December 2025 rankings, December 2025. URL: <https://www.digitalapplied.com/blog/llm-comparison-guide-december-2025>.
- [21] Google. Today we're releasing gemini 3 – our most intelligent model that helps you bring any idea to life., November 2025. URL: <https://blog.google/products/gemini/gemini-3/>.
- [22] Anthropic. Anthropic is an ai safety and research company that's working to build reliable, interpretable, and steerable ai systems. URL: <https://www.anthropic.com/news/claude-opus-4-5>.
- [23] OpenAI. Pricing information for the openai platform. URL: <https://platform.openai.com>.
- [24] MiniMaxIR. Most coders want ai to write code faster: I want ai to write faster code., January 2025. URL: <https://minimaxir.com/2025/01/write-better-code/>.
- [25] OpenAI. Gpt-5.1: A smarter, more conversational chatgpt, November 2025. URL: <https://openai.com/index/gpt-5-1/>.
- [26] Google Colab. URL: <https://colab.research.google.com/>.
- [27] Visual Studio Code. Visual studio code redefines ai-powered coding with github copilot for building and debugging modern web and cloud applications. visual studio code is free and available on your favorite platform - linux, macos, and windows. URL: <https://code.visualstudio.com/>.
- [28] Google Workspace. Google docs: Online document pdf editor. URL: <https://workspace.google.com/products/docs/>.
- [29] App Store. Download macos sonoma by apple on the app store. see screenshots, ratings and reviews, user tips, and more games like macos sonoma., 2025. URL: <https://apps.apple.com/us/app/macos-sonoma/id6450717509?mt=12>.

- [30] Google Chrome. Chrome is the official web browser from google, built to be fast, secure, and customizable. download now and make it yours. URL: <https://www.google.com/chrome/by-environment/chrome-browser/>.
- [31] OpenAI. Introducing gpt-5.2. the most advanced frontier model for professional work and long-running agents., December 2025. URL: <https://openai.com/index/introducing-gpt-5-2/>.
- [32] xAI. Grok 4.1 is now available to all users on grok.com, and the ios and android apps., 2025. URL: <https://x.ai/news/grok-4-1>.
- [33] xAI. Supergrok. introducing grok 4.1. the most powerful ai model., 2025. URL: <https://grok.com/plans>.
- [34] OpenAI. Developer quickstart. make your first api request in minutes. learn the basics of the openai platform., December 2025. URL: <https://platform.openai.com/docs/overview>.
- [35] Google. Gemini 2.5 is our most intelligent ai model, now with thinking., March 2025. URL: <https://blog.google/technology/google-deepmind/gemini-model-thinking-updates-march-2025/>.
- [36] Google AI Studio. What will you build? push gemini to the limits of what ai can do, powered by the gemini api. this experimental model is for feedback and testing only. no production use., 2025. URL: https://aistudio.google.com/prompts/new_chat.
- [37] Anthropic. Introducing claude sonnet 4.5: Claude sonnet 4.5 is the best coding model in the world., September 2025. URL: <https://www.anthropic.com/news/claude-sonnet-4-5>.
- [38] Anthropic. Announcements. introducing claude pro., September 2023. URL: <https://www.anthropic.com/news/claude-pro>.
- [39] DeepSeek. We introduce deepseek-v3.2, a model that harmonizes high computational efficiency with superior reasoning and agent performance., December 2025. URL: <https://huggingface.co/deepseek-ai/DeepSeek-V3.2>.
- [40] DeepSeek. The deepseek r1 model has undergone a minor version upgrade, with the current version being deepseek-r1-0528., November 2025. URL: <https://huggingface.co/deepseek-ai/DeepSeek-R1-0528>.
- [41] Fireworks Fastest Inference for Generative AI. Use state-of-the-art, open-source llms and image models at blazing fast speed, or fine-tune and deploy your own at no additional cost with fireworks ai! URL: <https://fireworks.ai>.
- [42] Kimi. Kimi k2 thinking is the latest, most capable version of open-source thinking model., November 2025. URL: <https://huggingface.co/moonshotai/Kimi-K2-Thinking>.
- [43] Kimi. We're on a journey to advance and democratize artificial intelligence through open source and open science., November 2025. URL: <https://huggingface.co/moonshotai/Kimi-K2-Instruct-0905>.
- [44] Z.ai. Compared with glm-4.5, glm-4.6 brings several key improvements., November 2025. URL: <https://huggingface.co/zai-org/GLM-4.6>.
- [45] MiniMax. Today, we release and open source minimax-m2, a mini model built for max coding agentic workflows. URL: <https://huggingface.co/MiniMaxAI/MiniMax-M2>.
- [46] OpenAI. Welcome to the gpt-oss series, openai's open-weight models designed for powerful reasoning, agentic tasks, and versatile developer use cases., August 2025. URL: <https://huggingface.co/openai/gpt-oss-120b>.
- [47] Qwen. Today, we're announcing qwen3-coder, our most agentic code model to date., 2025. URL: <https://huggingface.co/Qwen/Qwen3-Coder-480B-A35B-Instruct>.
- [48] Center for Drug Evaluation and Research. Fda adverse event reporting system (faers): Latest quarterly data files. FDA, April 2019. URL: <https://www.fda.gov/drugs/fdas-adverse-event-reporting-system-faers/fda-adverse-event-reporting-system-faers-latest-quarterly-data-files>.
- [49] Aman Madaan, Niket Tandon, Prakhar Gupta, Skyler Hallinan, Luyu Gao, Sarah Wiegrefe, Uri Alon, Nouha Dziri, Shrimai Prabhumoye, Yiming Yang, Shashank Gupta, Bodhisattwa Prasad Majumder, Katherine Hermann, Sean Welleck, Amir Yazdanbakhsh, and Peter Clark. Self-refine: Iterative refinement with self-feedback. *Advances in Neural Information Processing Systems*, 36:46534–46594, December 2023. URL: https://proceedings.neurips.cc/paper_files/paper/2023/hash/91edff07232fb1b55a505a9e9f6c0ff3-Abstract-Conference.html.
- [50] Mark Chen, Jerry Tworek, Heewoo Jun, Qiming Yuan, Henrique Ponde de Oliveira Pinto, Jared Kaplan, Harri Edwards, Yuri Burda, Nicholas Joseph, Greg Brockman, Alex Ray, Raul Puri, Gretchen Krueger, Michael Petrov, Heidy Khlaaf, Girish Sastry, Pamela Mishkin, Brooke Chan, Scott Gray, Nick Ryder, Mikhail Pavlov, Alethea Power, Lukasz Kaiser, Mohammad Bavarian, Clemens Winter, Philippe Tillet, Felipe Petroski Such, Dave Cummings, Matthias Plappert, Fotios Chantzis, Elizabeth Barnes, Ariel Herbert-Voss, William Hebggen Guss, Alex Nichol, Alex Paino, Nikolas Tezak, Jie Tang, Igor Babuschkin, Suchir Balaji, Shantanu Jain, William Saunders, Christopher Hesse, Andrew N. Carr, Jan Leike, Josh Achiam, Vedant Misra, Evan Morikawa, Alec Radford, Matthew Knight, Miles Brundage, Mira Murati, Katie Mayer, Peter Welinder, Bob McGrew, Dario Amodei, Sam McCandlish, Ilya Sutskever, and Wojciech Zaremba. Evaluating large language models trained on code. *arXiv*, (arXiv:2107.03374), 2021. arXiv:2107.03374. URL: <http://arxiv.org/abs/2107.03374>, doi:10.48550/arXiv.2107.03374.

- [51] Sewon Min, Xinxu Lyu, Ari Holtzman, Mikel Artetxe, Mike Lewis, Hannaneh Hajishirzi, and Luke Zettlemoyer. Rethinking the role of demonstrations: What makes in-context learning work? *arXiv*, (arXiv:2202.12837), October 2022. arXiv:2202.12837. URL: <http://arxiv.org/abs/2202.12837>, doi:10.48550/arXiv.2202.12837.
- [52] Chen Qian, Wei Liu, Hongzhang Liu, Nuo Chen, Yufan Dang, Jiahao Li, Cheng Yang, Weize Chen, Yusheng Su, Xin Cong, Juyuan Xu, Dahai Li, Zhiyuan Liu, and Maosong Sun. Chatdev: Communicative agents for software development. In Lun-Wei Ku, Andre Martins, and Vivek Srikumar, editors, *Proceedings of the 62nd Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers)*, page 15174–15186, Bangkok, Thailand, August 2024. Association for Computational Linguistics. URL: <https://aclanthology.org/2024.acl-long.810/>, doi:10.18653/v1/2024.acl-long.810.
- [53] Hung Le, Yue Wang, Akhilesh Deepak Gotmare, Silvio Savarese, and Steven Chu Hong Hoi. Coderl: Mastering code generation through pretrained models and deep reinforcement learning. *Advances in Neural Information Processing Systems*, 35:21314–21328, December 2022. URL: https://proceedings.neurips.cc/paper_files/paper/2022/hash/8636419dea1aa9fbd25fc4248e702da4-Abstract-Conference.html.
- [54] John Yang, Kilian Lieret, Joyce Yang, Carlos E. Jimenez, Ofir Press, Ludwig Schmidt, and Diyi Yang. Codeclash: Benchmarking goal-oriented software engineering. *arXiv*, (arXiv:2511.00839), November 2025. arXiv:2511.00839. URL: <http://arxiv.org/abs/2511.00839>, doi:10.48550/arXiv.2511.00839.
- [55] Emeri Potter, Melissa Reyes, Jennifer Naples, and Gerald Dal Pan. Fda adverse event reporting system (faers) essentials: A guide to understanding, applying, and interpreting adverse event data reported to faers. *Clinical Pharmacology Therapeutics*, 118(3):567–582, 2025. URL: <https://ascpt.onlinelibrary.wiley.com/doi/10.1002/cpt.3701>, doi:10.1002/cpt.3701.
- [56] Haiyan Yang, Zheng Wan, Moliang Chen, Xiaohong Zhang, Wugeng Cui, and Bin Zhao. A real-world data analysis of topotecan in the fda adverse event reporting system (faers) database. *Expert Opinion on Drug Metabolism Toxicology*, 19(4):217–223, April 2023. URL: <https://www.tandfonline.com/doi/full/10.1080/17425255.2023.2219390>, doi:10.1080/17425255.2023.2219390.
- [57] Roger J. Yu, Matthew S. Krantz, Elizabeth J. Phillips, and Cosby A. Stone. Emerging causes of drug-induced anaphylaxis: A review of anaphylaxis-associated reports in the fda adverse event reporting system (faers). *The Journal of Allergy and Clinical Immunology: In Practice*, 9(2):819–829.e2, February 2021. URL: <https://www.sciencedirect.com/science/article/pii/S2213219820309995>, doi:10.1016/j.jaip.2020.09.021.
- [58] Gemini. URL: <https://gemini.google/subscriptions/>.
- [59] Graziano Stefanelli. xai grok 4.1 subscriptions: Free access, supergrok plans, heavy tier, api pricing structure and use-case alignment, December 2025. URL: <https://www.datastudios.org/post/xai-grok-4-1-subscriptions-free-access-supergrok-plans-heavy-tier-api-pricing-structure-and-use>.
- [60] Kevin Kawchak. Ai peer review acceleration of llm-generated glioblastoma clinical trial patient matching ml, fda/ich/iso, and fastapi, November 2025. doi:10.5281/zenodo.17774560.
- [61] Kevin Kawchak. Llm-generated glioblastoma drug synergy machine learning: From rapid code prototypes to project deliverables package, November 2025. doi:10.5281/zenodo.17614396.
- [62] Kevin Kawchak. End-to-end oncology clinical trial llm efficiency for industry adoption with fda/ich regulations, October 2025. doi:10.5281/zenodo.17451709.
- [63] Kevin Kawchak. Code generation competition: 16 proprietary vs. open-source llms iterative learning based on fda adverse event reporting system, December 2025. doi:10.5281/zenodo.18029100.

11 Acknowledgments

The author would like to acknowledge Anthropic for providing access to Claude, Google for providing access to Gemini, OpenAI for providing access to ChatGPT and GPT, xAI for providing access to Grok, and Fireworks AI for providing access to DeepSeek, Kimi, GLM, MiniMax, OpenAI, and Qwen open-source LLMs.

12 Ethical disclosures

The author of the article declares no competing interests.

13 Rights and permissions

This article is distributed under the terms of the Creative Commons Attribution 4.0 International License (CC BY 4.0), which permits unrestricted use, distribution, and reproduction in any medium, provided the original author(s) and source are properly credited, a link to the Creative Commons license is provided, and any modifications made are indicated. To view a copy of this license, visit <https://creativecommons.org/licenses/by/4.0/>.

14 Cite this article

Kawchak K. Code Generation Competition: 16 Proprietary vs. Open-Source LLMs & Iterative Learning Based on FDA Adverse Event Reporting System. Zenodo. 2025; 10.5281/zenodo.18029100 [63].